

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY
VIZIANAGARAM
VIZIANAGARAM – 535003
ANDHRA PRADESH

FUEL CONSUMPTION TRACKER

By

PINNELLI AVINASH REDDY
(Regd. No.24VV1A1248)

Under the supervision of

MRS. MADHUMITA CHANDA

Department of Information Technology

Vizianagaram – 535003



JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY

VIZIANAGARAM

VIZIANAGARAM – 535003

ANDHRA PRADESH

Regd.No:25VV5A1272

This is to certify that this is a bonafide record of practical work done by P.KASI RAMA SUBRAMANYAM of III B.Tech I Semester Class in USER INTERFACEDESIGN USING FLUTTER Lab during the year 2025-26.

No.of Tasks Completed and Certified:

Lecture In-Charge **Head of The Department**

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY
JNTU-GURAJADA VIZIANAGARAM
COLLEGE OF ENGINEERING VIZIANAGARAM (A)
VIZIANAGARAM

Website:www.jntugvcev.edu.in

Subject Name: USER INTERFACE DESIGN USING FLUTTER Subject Code: R233112ES01

Academic Year: 2026 **Regulation: R23**

COURSE OUTCOMES

NBA Subject Code	Course Outcomes	
R233112ES01	CO1	Learns to Implement Flutter Widgets and Layouts.
	CO2	Understands Responsive UI Design and with Navigation in Flutter.
	CO3	Knowledge on Widgets and customize widgets for specific UI elements, Themes
	CO4	Understand to include animation apart from fetching data.

CO-PO Mapping

Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Outcomes		Program Outcomes (POs)														
		PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
R233112ES01	CO1	3	2	3	2	3	1	1	1	2	2	1	2	3	3	2
	CO2	3	3	3	2	3	1	2	2	2	3	2	2	3	3	2
	CO3	3	2	3	2	3	2	1	1	2	2	1	2	3	2	2
	CO4	3	3	3	3	3	2	1	2	3	3	2	3	3	3	3

Enter correlation levels 1,2 and 3 as defined below:

1: Slight (Low) 2: Moderate (Medium) 3: Substantial (High) If there is no correlation, put "-"

Signature of the Course Instructor

Index

S.No	Contents	Page no	Marks	Signature
1	Introduction	1		
2	Project Structure	2		
3	Database Setup (SQLite)	4		
4	Installation of Flutter (with PATH Setup)	6		
5	Installation of Dart SDK in Windows	10		
6	Installing Visual Studio Code	11		
7	Creating a New Flutter Project	12		
8	Dart Basic Programming	13		
9	Case Study on Fuel Consumption Tracker App	16		
10	Exploring Flutter Widgets	18		
11	Responsive UI Design	22		
12	Navigation Between Screens	26		
13	Stateful and Stateless Widgets	27		
14	Custom Widgets and Styling	30		
15	Forms and Validation	33		
16	Animations	39		
17	Local Data Layer and API Integration	44		
18	Testing and Debugging	47		
19	Work Flow Of Screens	49		
20	Conclusion	55		

1 Introduction

Flutter is an open-source software development framework created by Google. It allows developers to build high-quality, natively compiled applications for mobile, web, and desktop platforms from a single codebase. Flutter uses the Dart programming language and renders its user interface with its own high-performance graphics engine, so the same interface looks and behaves consistently on every supported platform.

Advantages of Flutter

- Cross-platform development with a single codebase.
- Hot reload for instant updates during development.
- A rich set of customizable, composable widgets.
- High performance because the application is compiled to native code.
- Strong support from Google and a large open-source community.
- Consistent Material Design across Android, iOS, web, and desktop.

Dart Language

Dart is the programming language used by Flutter. It is object-oriented, class-based, and supports both just-in-time (JIT) compilation for development and ahead-of-time (AOT) compilation for production. The syntax is simple and familiar to developers who have worked with Java, C++, or JavaScript.

Advantages of Dart

- Easy-to-learn, readable syntax.
- Sound null safety that prevents many runtime errors.
- Strong typing with type inference using the `var`, `final`, and `const` keywords.
- Asynchronous programming with `async` and `await`.
- JIT compilation for fast development and AOT compilation for optimized release builds.

Key Features of Flutter

- **Cross-Platform Development:** One codebase runs on Android, iOS, web, Windows, macOS, and Linux.
- **Hot Reload:** Code changes are reflected instantly without restarting the application, which speeds up testing and iteration.

- **Widget-Based Framework:** Everything in Flutter is a widget, including layouts, buttons, text, and images. Widgets are composed into trees to build complex interfaces.

- **High Performance:** Flutter applications are compiled to native machine code, which results in smooth animations and fast rendering.

- **Custom UI Design:** Material Design and Cupertino widgets allow flexible and visually appealing interfaces on every platform.

About the Present Project

The present project, **Fuel Consumption Tracker** (branded **FuelTrail**), is an offline-first Flutter application that helps a user record fuel fills, track the mileage of one or more vehicles, and monitor fuel expenses over time. The application stores all of its data locally using SQLite, so it works without an internet connection and without any backend service.

2 Project Structure

The Fuel Consumption Tracker is a Flutter-based application designed to record fuel purchases and compute fuel efficiency for one or more vehicles. A user can register an account or continue as a guest, add vehicles with their fuel type and tank capacity, and log each fuel fill with the date, odometer reading, fuel volume, total cost, and whether the tank was filled completely. From these logs the application calculates mileage using the full-tank-to-full-tank method and presents the results on a dashboard with charts and summary cards.

Below is the directory structure of the project:

```
fuel_consumption_tracker/
  lib/
    main.dart
    models/
      app_user.dart
      fuel_log.dart
      user_profile.dart
      vehicle.dart
    providers/
      fuel_provider.dart
      profile_provider.dart
      theme_provider.dart
    services/
      database_helper.dart
      database_helper_mobile.dart
      database_helper_web.dart
      database_viewer.dart
    screens/
      login_screen.dart
      register_screen.dart
      home_screen.dart
      dashboard_screen.dart
```

```
      add_log_screen.dart
      vehicles_screen.dart
      settings_screen.dart
      profile_screen.dart
      database_manager_screen.dart
    widgets/
      animated_card.dart
      animated_efficiency_gauge.dart
      auth_scaffold.dart
      dashboard_chart_toggle.dart
      empty_state.dart
      expandable_log_card.dart
      glass_container.dart
      hero_vehicle_widgets.dart
      mileage_gauge.dart
      stat_card.dart
    theme/
      app_theme.dart
  test/
    widget_test.dart
```

Description of Project Modules

- lib/ – The core directory of the Flutter application. It contains all Dart source files, organised into subfolders for modularity and clarity.

- models/ – Defines the data structures that represent the core entities of the application: AppUser, Vehicle, FuelLog, and UserProfile.

- providers/ – Contains the state management classes built on Provider and ChangeNotifier. FuelProvider manages vehicles, fuel logs, mileage records, and monthly expenses; ProfileProvider manages authentication and the user session; and ThemeProvider manages the light and dark themes.

- services/ – Contains the database layer. The helper files select the correct SQLite implementation for each platform, and the viewer file exposes the helper methods used by the built-in database manager.

- screens/ – Contains all the major user interface screens of the application. Each screen represents a key feature such as login, registration, the dashboard, adding a fuel log, or managing vehicles.

- widgets/ – Contains reusable user interface components such as glass containers, statistic cards, the mileage gauge, animated cards, and empty-state placeholders.

- theme/ – Contains the central application theme, including the light and dark colour schemes and the text theme.

- test/ – Contains widget tests that verify the startup flow, the login screen, and form validation.

The application follows a layered structure. The presentation layer consists of the screens and widgets, the state layer consists of the providers, and the data layer consists of the models and the database service. This separation keeps the code readable and makes each part easier to test and maintain.

3 Database Setup (SQLite)

Objective: To create a local, persistent database for storing users, vehicles, and fuel logs.

Relation to Project: The entire application is offline-first, so SQLite is the single source of truth for all data.

Why SQLite

The application does not use Firebase or any remote server. Instead, it uses the sqflite package, which provides a SQLite database on Android, iOS, macOS, and Windows, together with sqflite_common_ffi for desktop and Linux and sqflite_common_ffi_web for the web. This makes the application fully functional without a network connection.

The relevant dependencies in pubspec.yaml are:

```
sqflite: ^2.3.2
path_provider: ^2.1.4
path: ^1.8.0
sqflite_common_ffi: ^2.4.2+1
sqflite_common_ffi_web: ^1.1.2
```

Platform Selection

A single entry file selects the correct database implementation at compile time. On native platforms the IO implementation is used, and on the web the web implementation is used.

```
// lib/services/database_helper.dart
export 'database_helper_web.dart'
    if (dart.library.io) 'database_helper_mobile.dart';
```

Opening the Database

The database is opened once and cached for the lifetime of the application. The database file is named fuel_tracker.db and its schema version is 3.

```
class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._internal();
  static Database? _database;

  DatabaseHelper._internal() {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }

  Future<Database> get database async {
    _database ??= await _initDatabase();
    return _database!;
  }
```

```
  Future<Database> _initDatabase() async {
    final dbPath  = await getDatabasesPath();
    final path = join(dbPath, 'fuel_tracker.db');
    return await openDatabase(
      path,
      version: 3,
      onCreate: _createDb,
      onUpgrade: _upgradeDb,
    );
  }
}
```

Schema

Three tables are created. The users table stores account details together with the password salt and hash. The vehicles table stores one row per vehicle owned by a user. The fuel_logs table stores one row per fuel fill.

```
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  phone TEXT,
  salt TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  is_guest INTEGER NOT NULL DEFAULT 0
);

CREATE TABLE vehicles (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  name TEXT NOT NULL,
  fuel_type TEXT NOT NULL,
  tank_capacity REAL NOT NULL,
  FOREIGN KEY (user_id) REFERENCES users (id)
);

CREATE TABLE fuel_logs (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER NOT NULL,
  vehicle_id INTEGER NOT NULL,
  date TEXT NOT NULL,
  odometer REAL NOT NULL,
  fuel_volume REAL NOT NULL,
  total_cost REAL NOT NULL,
  is_full_tank INTEGER NOT NULL DEFAULT 0,
  FOREIGN KEY (user_id) REFERENCES users (id),
  FOREIGN KEY (vehicle_id) REFERENCES vehicles (id)
);
```

Indexes

The dashboard, the vehicles list, and the log list all filter fuel logs by user and vehicle and order them by date. Three indexes keep these queries fast as the history grows.

```
CREATE INDEX IF NOT EXISTS idx_vehicles_user
  ON vehicles(user_id);

CREATE INDEX IF NOT EXISTS idx_fuel_logs_user_vehicle_date
  ON fuel_logs(user_id, vehicle_id, date);

CREATE INDEX IF NOT EXISTS idx_fuel_logs_vehicle_date
  ON fuel_logs(vehicle_id, date);
```

Schema Migration

When an older database is opened, the upgrade callback adds the newer columns and indexes without deleting existing data.

```
Future<void> _upgradeDb(Database db, int oldVersion, int newVersion) async {
  if (oldVersion < 2) {
    await db.execute('''CREATE TABLE users (...)''');
    await db.execute('ALTER TABLE vehicles ADD COLUMN user_id INTEGER');
    await db.execute('ALTER TABLE fuel_logs ADD COLUMN user_id INTEGER');
  }
  if (oldVersion < 3) {
    await _createIndexes(db);
  }
}
```

4 Installation of Flutter (with PATH Setup)

Flutter is distributed as an SDK archive. The following steps describe how to install it on Windows and add it to the system PATH so that the flutter command can be used from any terminal.

Step 1: Download the Flutter SDK

1. Visit: https://docs.flutter.dev/get-started/install.
2. Choose the operating system (Windows, macOS, or Linux).
3. Download the Flutter SDK from the stable channel.
4. Extract the archive to a short, writable path such as C:\src\flutter.

Step 2: Add Flutter to the PATH

1. Open **Control Panel → System → Advanced system settings**.

2. Click **Environment Variables**.

3. Under **User variables** or **System variables**, select **Path** and click **Edit**.

4. Add the path to the `bin` directory of the SDK, for example `C:\src\flutter\bin`.

5. Confirm the dialog boxes and close them.

Step 3: Verify the Installation

Open a new Command Prompt or PowerShell window and run:

```
flutter --version
flutter doctor
```

The first command prints the installed Flutter and Dart versions. The second command checks the development environment and reports any missing tools.

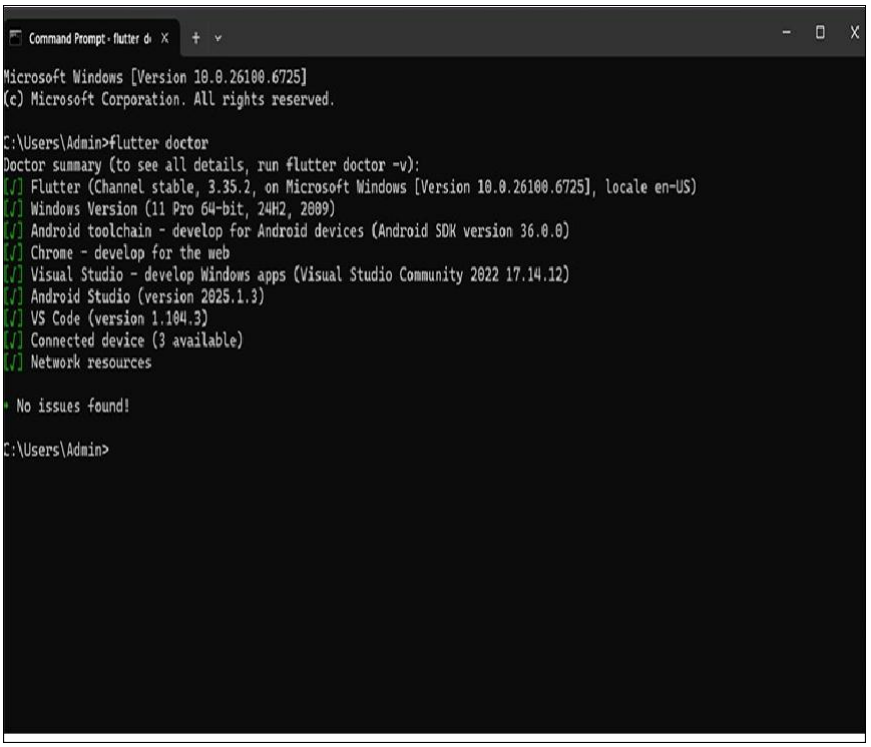


Figure 1: Flutter doctor output showing the development environment

Step 4: Install Android Studio

Although the application can be edited in any editor, Android Studio provides the Android SDK and the emulator that are required to run the application on Android.

1. Download Android Studio from https://developer.android.com/studio.

2. Install it using the default settings.

3. Open Android Studio and go to **File → Settings → Plugins**.

4. Install the **Flutter** plugin; the Dart plugin is installed automatically.

5. Open the **SDK Manager** and verify that the Android SDK location is set and that the required SDK platforms and build tools are installed.

Step 5: Create and Run an Emulator

1. Open Android Studio.

2. Go to **Tools → Device Manager**.

3. Click **Create Virtual Device**.

4. Choose a device model such as the Pixel 6.

5. Select a system image such as Android 13 or later.

6. Finish the wizard and launch the emulator.

Once the emulator is running, the Flutter application can be launched on it with the `flutter run` command.

Step 6: Accept the Android Licences

Before building for Android, the SDK licences must be accepted. Run the following command once on the machine and accept every licence that is shown.

```
flutter doctor --android-licenses
```

Step 7: Verify the Complete Toolchain

Run `flutter doctor` once more and confirm that every required section is marked as working. A healthy installation reports the following sections.

```
Flutter (Channel stable) ..... OK
Windows Version ..... OK
Android toolchain ..... OK
Chrome (web development) ..... OK
Visual Studio (Windows desktop development) ..... OK
Android Studio ..... OK
VS Code ..... OK
Connected device ..... OK
Network resources ..... OK
```

If a section is not marked OK, read the message printed below it and follow the suggested command. Most problems are caused by a missing PATH entry, an unaccepted licence, or a missing Android SDK platform.

Common Installation Problems

The following table lists the problems that were encountered while setting up this project and the steps that resolved them.

Message	Cause	Resolution
flutter is not recognised	PATH not applied	Reopen the terminal; verify the SDK bin folder is in PATH.
Android licence status unknown	Licences not accepted	Run flutter doctor –android-licenses.
Unable to locate Android SDK	SDK path missing	Set the SDK path in Android Studio or in local.properties.
No connected devices	Emulator not started	Launch a virtual device or enable USB debugging.
Gradle build failed	Corrupt cache	Run flutter clean and rebuild the project.

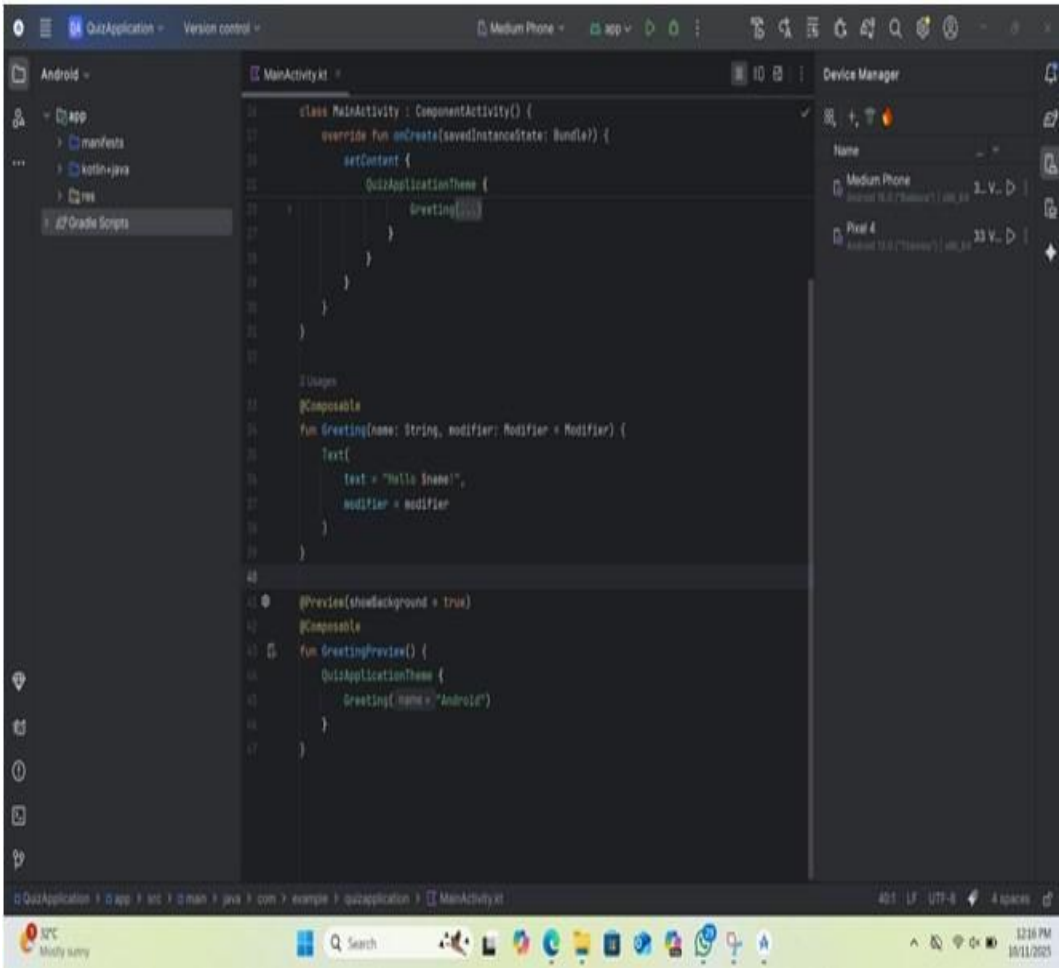


Figure 2: Android Studio with the Flutter and Dart plugins installed

5 Installation of Dart SDK in Windows

Dart is a programming language developed by Google, mainly used for building mobile, desktop, web, and server applications. Flutter already bundles a Dart SDK, but a standalone Dart SDK can also be installed for running Dart programs from the command line. The following steps describe the installation on Windows.

1. **Download the Dart SDK:** Go to https://dart.dev/get-dart, select the Windows section, choose the 64-bit ZIP archive, and download it.

2. **Extract the SDK:** Extract the ZIP archive to a preferred location, for example C:\dart-sdk.

3. **Set the environment variable:** Right-click **This PC → Properties → Advanced system settings → Environment Variables**. Under **System variables**, select **Path**, click **Edit**, and add the path to the bin folder inside the SDK, for example C:\dart-sdk\bin.

4. **Verify the installation:** Open a new Command Prompt and run the follow-

Once the Dart SDK is installed and configured, Dart programs can be compiled and run directly, and Flutter applications can use the bundled or standalone SDK without further configuration.

Running a Dart Program

A Dart file is executed with the dart run command. The small program below prints a line to the terminal and confirms that the toolchain works.

```
void main() {
  print('Fuel  Consumption  Tracker');
}

dart run hello.dart
```

6 Installing Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful open-source code editor developed by Microsoft. It is widely used for Flutter and Dart development because of its support for extensions, debugging, IntelliSense, and integration with the Flutter SDK.

Steps to Install VS Code

1. **Download VS Code:** Visit https://code.visualstudio.com/ and download the version suitable for the operating system.

2. **Install VS Code:** Run the downloaded installer and complete the installation using the default settings. This ensures that the `code` command is added to the PATH.

3. **Install extensions:** Launch VS Code, open the Extensions panel with **Ctrl + Shift + X**, and install the **Flutter** and **Dart** extensions.

4. **Create a new Flutter project:** Open the Command Palette with **Ctrl + Shift + P**, search for **Flutter: New Project**, choose the application type, enter a project name, and select a destination folder.

5. **Open the project:** VS Code opens the generated folder automatically. The `lib/`, `android/`, `ios/`, and other platform folders can then be explored.

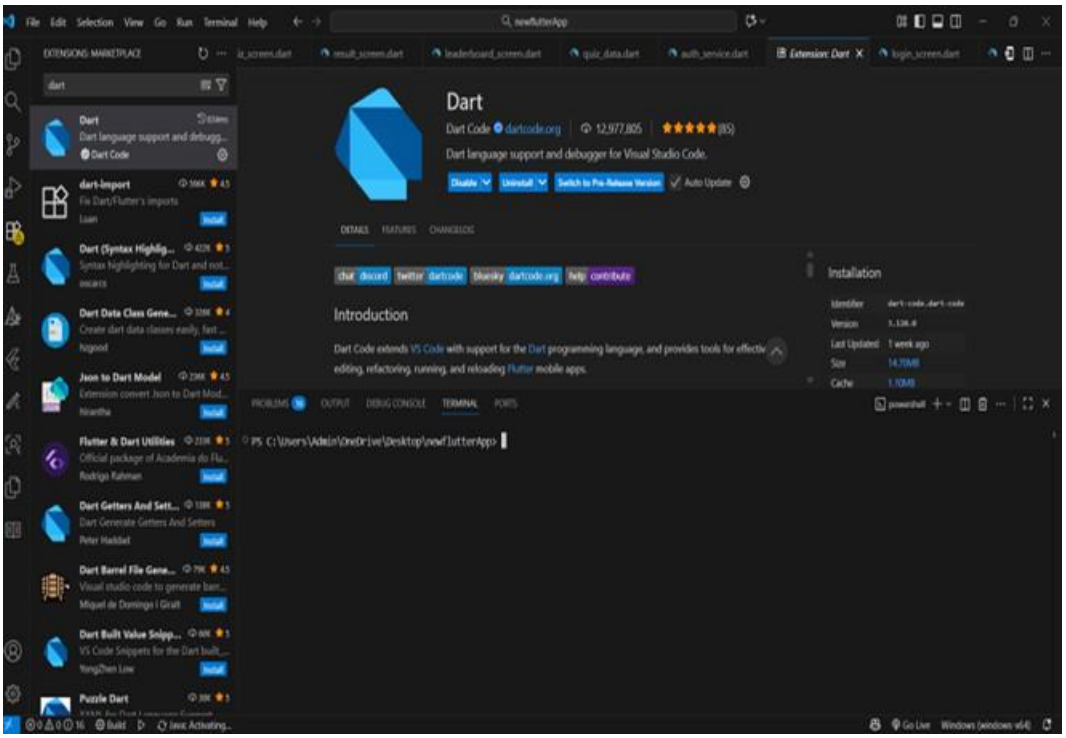


Figure 3: Installing the Flutter and Dart extensions in Visual Studio Code

7 Creating a New Flutter Project

Before creating a new Flutter project, ensure that the Flutter SDK is installed and added to the system PATH. A new project can be created either from the command line or from VS Code.

Steps to Create a New Flutter Project

1. Open a Command Prompt or Terminal.
2. Navigate to the desired folder.
3. Create a new Flutter project with the following command.

```
flutter create fuel_consumption_tracker
```

4. Navigate into the project directory.

```
cd fuel_consumption_tracker
```

5. Open the project in VS Code or another preferred editor.
6. Run the application on the connected device or emulator.

```
flutter run
```

Project Structure

After running the commands, Flutter generates the following structure:

- lib/ – Contains the main Dart source code.
- android/, ios/, web/, windows/, macos/, linux/ – Platform-specific projects.
- pubspec.yaml – The configuration file for dependencies and assets.
- test/ – Contains unit and widget tests.

The application code for this project is organised inside lib/ using the model, provider, service, screen, widget, and theme folders described in Section 2.

Project Dependencies

The generated pubspec.yaml is edited to declare the packages used by the application. Each dependency is resolved once and locked so that every build uses the same versions.

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  provider: ^6.1.2
  sqflite: ^2.3.2
  path_provider: ^2.1.4
  path: ^1.8.0
  fl_chart: ^0.66.0
  intl: ^0.19.0
  google_fonts: ^6.2.1
  shared_preferences: ^2.2.3
  sqflite_common_ffi: ^2.4.2+1
  sqflite_common_ffi_web: ^1.1.2
  crypto: ^3.0.7
```

After the file is saved, flutter pub get downloads the packages and updates the lock file. The application is then ready to import them.

8 Dart Basic Programming

Dart is a modern, object-oriented programming language developed by Google. It is optimized for building client applications, particularly mobile applications using the Flutter framework. Dart provides strong typing, asynchronous programming, and a rich standard library, which make it both easy to learn and powerful for application development.

Variables

Variables are containers that hold data values. Every variable in Dart has a name, a type, and a value. Dart is statically typed, but it also supports type inference.

```
void main() {
  String name = 'FuelTrail';
  int vehicleCount = 2;
double tankCapacity = 42.5;
bool isFullTank = true;

  var mileage = 18.4;       // inferred as double
  final createdAt = '2026-01-01'; // set once
  const maxLogs = 500;       // compile-time constant

  print('Name: $name');
  print('Vehicles: $vehicleCount');
  print('Tank: $tankCapacity L');
  print('Full tank: $isFullTank');
}
```

Conditional Statements

`

Conditional statements make decisions in a program. Dart provides if, else if, and else, as well as the conditional operator.

```
void main() {
  double mileage = 17.5;

  if (mileage >= 20) {
    print('Excellent efficiency');
  } else if (mileage >= 15) {
    print('Good efficiency');
  } else {
    print('Consider a service check');
  }
}
```

Loops

Loops repeat a block of code while a condition is true. Dart supports for, while, and do-while loops.

```
void main() {
  for (int i = 1; i <= 5; i++) {
    print('Log entry: $i');
  }
}
```

Lists

A list is an ordered collection of elements. Lists can grow dynamically and support random access by index.

```
void main() {
  List<String> fuelTypes = ['Petrol', 'Diesel', 'CNG'];
  fuelTypes.add('Electric');
  print('Fuel types: $fuelTypes');}
```

Maps

Maps store data as key-value pairs. They are useful for representing structured records and for grouping values.

```
void main() {
  Map<String, double> expenses = {
    '2026-01': 2400.0,
    '2026-02': 1850.0,
    '2026-03': 2100.0,
  };

  expenses.forEach((month, amount) {
    print('$month: Rs. $amount');
  });
}
```

Functions

Functions are reusable blocks of code that perform a specific task. They can take parameters and return values.

```
double calculateMileage(double distance, double fuel) {
  if (fuel <= 0) return 0;
  return distance / fuel;
}

void main() {
  final mileage = calculateMileage(300, 18.5);
  print('Mileage: ${mileage.toStringAsFixed(2)} km/L');
}
```

Classes and Objects

Dart is class-based. A class defines the properties and behaviour of an object. The models in this project are simple classes with named constructors and copy methods.

```
class FuelLog {
  final int? id;
  final int vehicleId;
  final String date;
  final double odometer;
  final double fuelVolume;
  final double totalCost;
  final bool isFullTank;

  FuelLog({
    this.id,
    required this.vehicleId,
    required this.date,
    required this.odometer,
    required this.fuelVolume,
    required this.totalCost,
    required this.isFullTank,
```

```
  });

  Map<String, dynamic> toMap() {
    return {
      'vehicle_id': vehicleId,
      'date': date,
      'odometer': odometer,
      'fuel_volume': fuelVolume,
      'total_cost': totalCost,
      'is_full_tank': isFullTank ? 1 : 0,
    };
  }
}
```

Null Safety

Dart is null safe, so a variable cannot hold null unless its type ends with a question mark. The null-aware operators and the null-coalescing operator make optional values safe to use.

```
String?  nickname;
final displayName = nickname ?? 'Guest';
final length = nickname?.length ?? 0;

late final TextEditingController controller;
controller = TextEditingController();
```

Asynchronous Programming

Database operations take time, so they are asynchronous. Dart uses Future together with async and await to write asynchronous code that reads like synchronous code.

```
Future<void> loadLogs() async {
  try {
    final logs = await database.getFuelLogs(userId: 1);
    print('Loaded ${logs.length} logs');
  } catch (e) {
    print('Failed to load logs: $e');
  }
}
```

9 Case Study on Fuel Consumption Tracker App

The Fuel Consumption Tracker is an interactive mobile application built using Flutter and Dart. It allows users to record fuel purchases, track the fuel efficiency of each vehicle, and compare monthly fuel expenses. In many households, fuel is a recurring expense, yet there is no simple way to know whether a vehicle is returning the expected mileage or whether running costs are rising over time. This project addresses that gap with a simple, private, and offline-first application.

Key Features

- **User Authentication:** Offline registration and sign-in with salted password hashing, plus a guest mode.
- **Vehicle Management:** Add, edit, select, and delete vehicles with their fuel type and tank capacity.
- **Fuel Logging:** Record the date, odometer, fuel volume, total cost, and full-tank status for each fill.
- **Mileage Calculation:** Compute mileage using the full-tank-to-full-tank method and display cost per kilometre.
- **Dashboard:** Show the latest mileage, total distance, total fuel, and total cost, with line and bar charts.
- **Monthly Expenses:** Group fuel spending by month.
- **Database Manager:** Inspect the schema, DDL, rows, and run SQL queries from inside the application.
- **Themes:** Light and dark themes with a persistent preference.
- **Offline Operation:** All data is stored locally in SQLite.

Technologies Used

- **Flutter SDK:** For building the cross-platform user interface.
- **Dart:** The core programming language.
- **Provider:** For state management with ChangeNotifier.
- **SQLite:** Through the sqflite family of packages for native, desktop, and web storage.
- **SharedPreferences:** For persisting the session and theme.
- **fl_chart:** For the mileage and expense charts.
- **google_fonts:** For the Inter typeface.
- **crypto:** For SHA-256 password hashing.
- **VS Code:** As the development environment.

Expected Outcome

The outcome is a complete, self-contained application that allows a user to record fuel data and immediately see meaningful efficiency and cost information. Because the application does not depend on a server, it remains usable without connectivity and keeps the user's data private on the device.

10 Exploring Flutter Widgets

Objective: To explore the basic widgets such as Text, Container, Card, and ListView.

Relation to Project: These widgets are used throughout the application for the dashboard cards, the fuel log list, and the navigation shell.

Description

Flutter builds the interface by composing widgets into a tree. The root widget in this project is a MaterialApp, which is wrapped by the providers. The signed-in state determines whether the login screen or the home screen is shown. The home screen hosts a NavigationBar that switches between the dashboard, vehicles, settings, and profile screens.

```
MaterialApp
  MultiProvider
    HomeScreen
      Scaffold
       AppBar
       Body
          DashboardScreen
            VehicleSelector
            MileageGauge
            StatCard
            ChartToggle
            ExpandableLogCard
       NavigationBar
          Dashboard | Vehicles | Settings | Profile
```

Widgets Used

Widget	Purpose in the Project
Scaffold	Provides the basic page structure with an app bar and body.
AppBar	Displays the screen title and actions.
Column	Stacks the dashboard sections vertically.
Row	Arranges summary values and icons horizontally.
Card	Groups related information with rounded corners.
ListView	Displays the fuel log history efficiently.
NavigationBar	Provides bottom navigation between main screens.
TextField	Accepts user input in forms.
DropdownButton	Selects the active vehicle and fuel type.
TweenAnimationBuilder	Animates the gauge and card entrances.

Widget	Purpose in the Project
CustomPaint	Draws the semicircular mileage gauge.
FloatingActionButton	Opens the add-log screen.

Example: A Reusable Statistic Card

The dashboard uses a small reusable widget to display a label, a value, and an icon. This keeps the dashboard code short and consistent.

```
class StatCard extends StatelessWidget {
  final String label;
  final String value;
  final IconData icon;
  final Color? accent;

  const StatCard({
    super.key,
    required this.label,
    required this.value,
    required this.icon,
    this.accent,
  });

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Row(
          children: [
            Icon(icon, color: accent ?? theme.colorScheme.primary),
            const SizedBox(width: 12),
            Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(label, style: theme.textTheme.bodySmall),
                Text(value, style: theme.textTheme.titleMedium),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

Building a Tree with the build Method

Every widget implements a build method that returns another widget. A screen is therefore a tree of small widgets rather than one large block of code. The method below composes a title, a spacer, and an expanding list.

```
@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const SectionTitle('Recent Fuel Logs'),
        const SizedBox(height: 8),
        Expanded(child: _logList()),
      ],
    ),
  );
}
```

Displaying a Scrollable List

The fuel log history is rendered with ListView.builder, which creates rows lazily as they scroll into view. Each row is an ExpandableLogCard that shows the summary and expands to reveal the detailed values.

```
ListView.builder(
  itemCount: logs.length,
  padding: const EdgeInsets.only(bottom: 96),
  itemBuilder: (context, index) {
    final log = logs[index];
    return ExpandableLogCard(
      log: log,
      onDelete: () {
        context.read<FuelProvider>().deleteFuelLog(log.id!);
      },
    );
  },
);
```

Collecting Text Input

Text fields use a controller to read the current value and an input decoration to describe the expected content. The decoration also supplies the label, the icon, and the border style.

```
final _searchController = TextEditingController();

TextField(
  controller: _searchController,
  decoration: InputDecoration(
    labelText: 'Search vehicle',
    prefixIcon: const Icon(Icons.search),
```

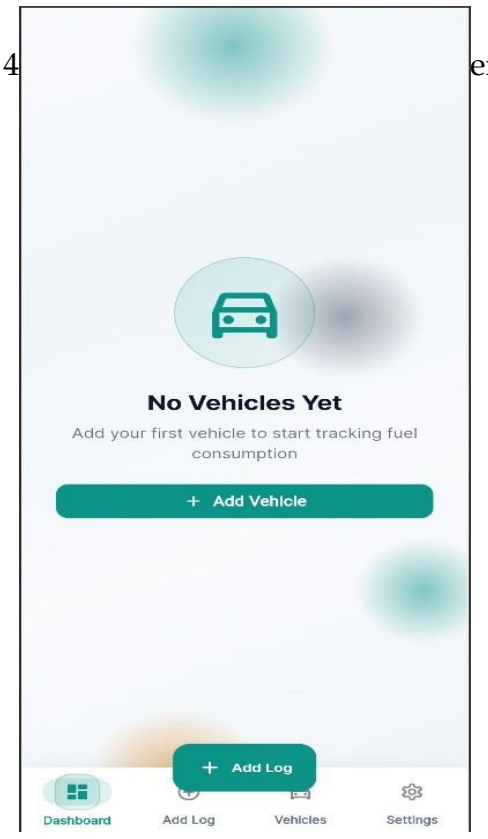
```
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(14),
    ),
  ),
  onChanged: (value) => setState(() {}),
);
```

Styling with Container

A Container combines margins, padding, size, and decoration. It is used for the small bordered panels that group related values, such as the reference odometer and the fuel consumed between two full tanks.

```
Container(
  margin: const EdgeInsets.symmetric(vertical: 6),
  padding: const EdgeInsets.all(14),
  decoration: BoxDecoration(
    color: theme.colorScheme.surface,
    borderRadius: BorderRadius.circular(16),
    border: Border.all(color: theme.dividerColor),
  ),
  child: child,
);
```

Output:

Figure 4 

example

11 Responsive UI Design

Objective: To build an interface that adapts to different screen sizes and orientations.

Relation to Project: The application runs on phones, tablets, and desktop, so the dashboard and forms must reflow gracefully.

Description

Flutter provides MediaQuery to read the size of the current screen and LayoutBuilder to build a layout based on the constraints given by the parent. The project uses both to switch between a single-column layout on narrow screens and a multi-column layout on wider screens. Safe areas are respected so that content is not hidden behind notches or system bars.

```
class DashboardLayout extends StatelessWidget {
  final Widget gauge;
  final List<Widget> cards;

  const DashboardLayout({
    super.key,
    required this.gauge,
    required this.cards,
  });

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        final isWide = constraints.maxWidth >= 700;
        if (isWide) {
          return Row(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Expanded(flex: 2, child: gauge),
              Expanded(flex: 3, child: Column(children: cards)),
            ],
          );
        }
        return Column(children: [gauge, ...cards]);
      },
    );
  }
}
```

Glassmorphism Containers

The visual identity of the application is based on translucent, rounded cards over a soft gradient background. A reusable GlassContainer widget applies the blur, border, and background tint in one place.

```
class GlassContainer extends StatelessWidget {
  final Widget child;
  final EdgeInsets padding;
  final double radius;

  const GlassContainer({
    super.key,
    required this.child,
    this.padding = const EdgeInsets.all(16),
    this.radius = 18,
  });

  @override
  Widget build(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return Container(
      padding: padding,
      decoration: BoxDecoration(
        color: isDark
            ? Colors.white.withValues(alpha: 0.06)
            : Colors.white.withValues(alpha: 0.7),
        borderRadius: BorderRadius.circular(radius),
        border: Border.all(
          color: Colors.white.withValues(alpha: isDark ? 0.08 : 0.8),
        ),
      ),
      child: child,
    );
  }
}
```

Reading the Screen Size

MediaQuery exposes the size, the orientation, and the text scaling of the current screen. These values are read once per build and used to choose the number of columns.

```
final media = MediaQuery.of(context);
final width = media.size.width;
final height = media.size.height;
final isLandscape = media.orientation == Orientation.landscape;
final textScale = media.textScaler.scale(1.0);

final columns = width >= 900
    ? 4
    : width >= 600
        ? 2
        : 1;
```

Breakpoints Used in the Project

The application uses three breakpoints. They are applied to the dashboard, the vehicle grid, and the summary cards.

Width	Layout	Typical device
Below 600 px	Single column, full-width cards	Phone in portrait
600–899 px	Two columns, cards side by side	Tablet in portrait
900 px and above	Gauge beside the cards, four columns	Tablet landscape or desktop

Accessibility and Safe Areas

A SafeArea keeps content away from notches and system bars. A SingleChildScrollView allows tall forms to scroll when the keyboard is open. Text scaling is respected because every text style comes from the theme instead of a fixed font size, so a user who increases the system font size still sees readable content.

```
SafeArea(
  child: SingleChildScrollView(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        const SizedBox(height: 12),
        const SectionTitle('Add Fuel Log'),
        const VehicleSelector(),
      ],
    ),
  ),
);
```

Output:

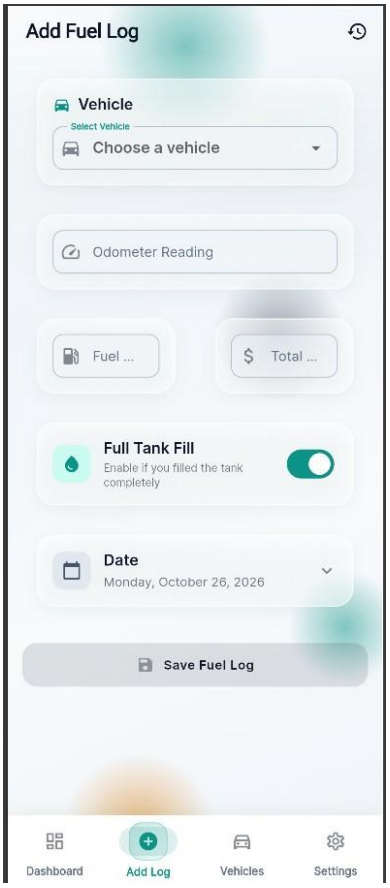


Figure 5: Responsive dashboard on a wide screen

12 Navigation Between Screens

Objective: To move between screens and pass data when required.

Relation to Project: Navigation connects the authentication flow, the main shell, and the add/edit forms.

Description

The application uses Navigator with MaterialPageRoute for screen-to-screen navigation. The root widget chooses the login screen or the home screen based on the authentication state, which acts as an authentication gate. Inside the home screen, a NavigationBar switches between the four main destinations without rebuilding the whole application.

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (_) => const AddLogScreen(),
  ),
);
```

Returning to the previous screen is done with:

```
Navigator.pop(context);
```

Passing Data Back

A route can also return a value to the screen that opened it. The add-log route returns true after a successful save so that the dashboard can refresh.

```
final saved = await Navigator.push<bool>(
  context,
  MaterialPageRoute(builder: (_) => const AddLogScreen()),
);
if (saved == true) {
  await context.read<FuelProvider>().reload();
}
```

Authentication Gate

The root widget listens to the profile provider and decides which screen to show. When the user signs out, the provider notifies its listeners and the login screen is shown again.

```
MaterialApp(
  title: 'Fuel Consumption Tracker',
  debugShowCheckedModeBanner: false,
  theme: _appLightTheme,
  darkTheme: _appDarkTheme,
  themeMode: themeMode,
  home: profileProvider.isLoggedIn
      ? const HomeScreen()
      : const LoginScreen(),
);
```

Bottom Navigation

The home screen keeps an index in its state and renders the selected destination.

```
NavigationBar(
  selectedIndex: _index,
  onDestinationSelected: (value) {
    setState(() => _index = value);
class _CounterExampleState extends State<CounterExample> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Count: $_count'),
        ElevatedButton(
          onPressed: () => setState(() => _count++),
          child: const Text('Increase'),
        ),
      ],
    );
  }
}
```

Shared State with Provider

Application-wide state is not kept inside individual widgets. Instead it lives in a ChangeNotifier and is exposed through Provider. Widgets listen to the provider and rebuild when it notifies them.

```
class ThemeProvider extends ChangeNotifier {
  ThemeMode _themeMode;

  ThemeProvider(bool initialDarkMode)
      : _themeMode = initialDarkMode ? ThemeMode.dark : ThemeMode.light;

  ThemeMode get themeMode => _themeMode;
  bool get isDarkMode => _themeMode == ThemeMode.dark;

  Future<void> toggleTheme() async {
    _themeMode = isDarkMode ? ThemeMode.light : ThemeMode.dark;
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool('isDarkMode', !isDarkMode);
    notifyListeners();
  }
}
```

13 Stateful and Stateless Widgets

Objective: To understand the difference between stateless and stateful widgets and to use each type in the appropriate part of the application.

Relation to Project: Presentational widgets such as StatCard are stateless, while interactive widgets such as the add-log form, the animated cards, and the dashboard

shell are stateful because their contents change in response to user input.

Description

A widget is an immutable description of part of the user interface. Flutter composes these descriptions into a tree and rebuilds the affected parts when the underlying data changes. Widgets are divided into two categories according to whether they own mutable state. A stateless widget receives all of its data through its constructor and never changes it. A stateful widget owns a State object that persists between rebuilds and is updated with setState.

Stateless Widgets

A StatelessWidget is used when the appearance of the widget depends only on the values passed to it. The build method is called whenever the parent rebuilds and returns the same interface for the same inputs. Stateless widgets are cheap to create, easy to test, and free of lifecycle concerns.

The StatCard used on the dashboard is a typical example. It receives a label, a value, and an icon, and renders them without storing anything of its own.

```
class StatCard extends StatelessWidget {
  const StatCard({super.key, required this.label,
      required this.value, required this.icon});
  final String label;
  final String value;
  final IconData icon;
  @override
  Widget build(BuildContext context) => Card(
        child: ListTile(leading: Icon(icon),
          title: Text(label), subtitle: Text(value)));
}
```

Because the widget keeps no state, a new StatCard can be created for every value on the dashboard without any risk of stale data.

Stateful Widgets

A StatefulWidget is used when the widget must remember information between rebuilds, such as the current text in a field, the selected tab, or the result of an asynchronous request. The widget itself stays immutable, but it creates a State object that survives rebuilds.

The add-log screen is stateful. Its state holds the form key, the controllers, the selected vehicle, the chosen date, and a saving flag so that the form can show progress and refresh after a successful save.

```
class AddLogScreen extends StatefulWidget {
  const AddLogScreen({super.key});
  @override
  State<AddLogScreen> createState() => _AddLogScreenState();
}

class _AddLogScreenState extends State<AddLogScreen> {
  final _formKey = GlobalKey<FormState>();
  bool _saving = false;
  // controllers and selected values
}
```

setState and Rebuilding

State is changed through setState. The callback updates the fields of the State object and then asks Flutter to rebuild the widget. Only the subtree below the stateful widget is

rebuilt, so the rest of the interface is left untouched.

```
void _increment() {
  setState(() {
    _count++;
  });
}
```

setState must not be called after the widget has been disposed, and asynchronous work should check mounted before updating the state. Expensive work is performed outside the callback so that the rebuild itself stays fast.

Comparison

The difference between the two widget types is summarised below.

Aspect	StatelessWidget	StatefulWidget
Mutable state	None	Managed inside the State object
Rebuild trigger	Parent rebuild	setState or parent rebuild
Lifecycle	build only	initState, didUpdateWidget, dispose
Typical use	Labels, cards, icons	Forms, navigation shell, toggles
Project examples	StatCard, SectionTitle	AddLogScreen, HomeScreen

Lifecycle of a Stateful Widget

The state object is created once and lives as long as the widget remains in the tree. Controllers are created in initState and released in dispose, which prevents memory leaks when the screen is closed.

```
class _AddLogScreenState extends State<AddLogScreen> {
  late final TextEditingController _odometerController;
  late final  TextEditingController _fuelController;

  @override
  void initState() {
    super.initState();
    _odometerController  =  TextEditingController();
    _fuelController = TextEditingController();
  }

  @override
  void dispose() {
    _odometerController.dispose();
    _fuelController.dispose();
    super.dispose();
  }
}
```

14 Custom Widgets and Styling

Objective: To build reusable, styled components that give the application a consistent identity.

Relation to Project: Custom widgets such as GlassContainer, StatCard, MileageGauge, and AnimatedCard are reused across the dashboard, the forms, and the authentication screens.

Theme

The application defines a complete light and dark theme in lib/theme/app_theme.dart. Material 3 is enabled, the Inter font is used through google_fonts, and the colour scheme is built around a teal primary colour with amber and slate accents.

Role	Light	Dark
Primary	#0D9488	#5EEAD4
Primary container	#CCFBF1	#134E4A
Secondary	#475569	#94A3B8
Tertiary	#D97706	#FCD34D
Background	#F8FAFC	#0F172A
Surface	#FFFFFF	#1E293B
Error	#DC2626	#FCA5A5
Success	#059669	#6EE7B7
Warning	#D97706	#FCD34D

The Mileage Gauge

The most distinctive custom widget is the semicircular mileage gauge. It is implemented with CustomPaint and a painter that draws a track arc, a gradient value arc, tick marks, a needle, a hub, and the numeric readout. The needle sweeps to its value with an animation.

```
class _MileageGaugePainter extends CustomPainter {
  final double fraction;
  // ... text painters and colours ...

  static const double _sweep = math.pi;        // 180 degrees
  static const double _start = math.pi;        // start from the left

  @override
  void paint(Canvas canvas, Size size) {
    final center = Offset(size.width / 2, size.height - 10);
    final radius = math.min(size.width / 2 - 14, size.height - 34);
    final arcRect = Rect.fromCircle(center: center, radius: radius);

    final trackPaint = Paint()
      ..style = PaintingStyle.stroke
      ..strokeWidth = 13
      ..strokeCap = StrokeCap.round
      ..color = trackColor;
    canvas.drawArc(arcRect, _start, _sweep, false, trackPaint);

    if (fraction > 0) {
      final valuePaint = Paint()
        ..style = PaintingStyle.stroke
        ..strokeWidth = 13
        ..strokeCap = StrokeCap.round
        ..shader = const SweepGradient(
          startAngle: math.pi,
          endAngle: math.pi * 2,
            colors: [Color(0xFFD97706), Color(0xFF0D9488), Color(0xFF475569)],
          ).createShader(Rect.fromCircle(center: center, radius: radius + 8));
      canvas.drawArc(arcRect, _start, _sweep * fraction, false, valuePaint);
    }
```

```
  }

  @override
  bool shouldRepaint(covariant _MileageGaugePainter oldDelegate) {
    return oldDelegate.fraction != fraction;
  }
}
```

Output:



15 Forms and Validation

Objective: To collect user input safely and reject invalid values before they reach the database.

Relation to Project: The login, registration, vehicle, and fuel-log screens all use forms with validators.

Description

Flutter forms are built with Form and TextFormField. A GlobalKey<FormState> is used to validate all fields at once. Each field has a validator that returns an error message or null. Numeric fields parse the text with double.tryParse and reject empty, negative, or non-numeric values.

Login Form Validation

The login screen checks that the email and password are not empty before calling the provider.

```
final _formKey = GlobalKey<FormState>();
TextFormField(
  controller: _emailController,
  keyboardType: TextInputType.emailAddress,
  decoration: const InputDecoration(labelText: 'Email'),
  validator: (value) {
    if (value == null || value.trim().isEmpty) {
      return 'Enter your email';
    }
    if (!value.contains('@')) {
      return 'Enter a valid email';
    }
    return null;
  },
);

TextFormField(
  controller: _passwordController,
  obscureText: true,
  decoration: const InputDecoration(labelText: 'Password'),
  validator: (value) {
    if (value == null || value.isEmpty) {
      return 'Enter your password';
    }
    if (value.length < 6) {
      return 'Password must be at least 6 characters';
    }
    return null;
  },
);
```

Registration Form Validation

The registration screen checks the name, the email address, the password, and the confirmation password. The confirmation field compares its value with the password controller, so both entries must match before the account is created.

```
TextFormField(
  controller: _nameController,
  textCapitalization: TextCapitalization.words,
  decoration: const InputDecoration(labelText: 'Full name'),
  validator: (value) {
    if (value == null || value.trim().length < 3) {
      return 'Enter your full name';
    }
    return null;
  },
);

TextFormField(
  controller: _confirmController,
  obscureText: true,
  decoration: const InputDecoration(labelText: 'Confirm password'),
  validator: (value) {
    if (value != _passwordController.text) {
      return 'Passwords do not match';
    }
    return null;
  },
);
```

Validation Rules

The validators used across the application follow a small set of rules. Keeping the rules in one place makes the error messages consistent.

Field	Rule	Message
Full name	At least three characters	Enter your full name
Email	Contains @ and a domain	Enter a valid email
Password	At least six characters	Password too short
Confirmation	Equals the password	Passwords do not match
Odometer	Non-negative number	Enter a valid odometer reading
Fuel volume	Number greater than zero	Enter a fuel volume greater than zero
Total cost	Non-negative number	Enter a valid cost
Tank capacity	Number greater than zero	Enter a valid capacity

Restricting Numeric Input

The keyboard type alone does not stop invalid characters, so input formatters restrict the accepted characters at the point of entry.

```
TextFormField(
  controller: _odometerController,
  keyboardType: const TextInputType.numberWithOptions(decimal: true),
  inputFormatters: [
    FilteringTextInputFormatter.allow(RegExp(r'[0-9.]')),
  ],
  decoration: const InputDecoration(labelText: 'Odometer (km)'),
);
```

Fuel Log Form Validation

The add-log screen validates the odometer, fuel volume, and total cost. The full-tank switch determines whether the entry can be used as a mileage reference.

```
TextFormField(
  controller: _odometerController,
  keyboardType: const TextInputType.numberWithOptions(decimal: true),
  decoration: const InputDecoration(labelText: 'Odometer (km)'),
  validator: (value) {
    final parsed = double.tryParse(value ?? '');
    if (parsed == null) return 'Enter a valid odometer reading';
    if (parsed < 0) return 'Odometer cannot be negative';
```

```
    return null;
  },
);

TextFormField(
  controller:  _fuelController,
  keyboardType: const TextInputType.numberWithOptions(decimal: true),
  decoration: const InputDecoration(labelText: 'Fuel volume (L)'),
  validator: (value) {
    final parsed = double.tryParse(value ?? '');
    if (parsed == null || parsed <= 0) {
      return 'Enter a fuel volume greater than zero';
    }
    return null;
  },
);
```

Vehicle Form

A vehicle is created from a short form with a name, a fuel type, and a tank capacity. The fuel type is selected from a dropdown so that only the supported values can be stored.

```
DropdownButtonFormField<String>(
  value: _fuelType,
  decoration: const InputDecoration(labelText: 'Fuel type'),
  items: const [
    DropdownMenuItem(value: 'Petrol', child: Text('Petrol')),
    DropdownMenuItem(value: 'Diesel', child: Text('Diesel')),
    DropdownMenuItem(value: 'CNG', child: Text('CNG')),
    DropdownMenuItem(value: 'Electric', child: Text('Electric')),
  ],
  onChanged: (value) => setState(() => _fuelType = value!),
  validator: (value) => value == null ? 'Select a fuel type' : null,
);

TextFormField(
  controller:  _capacityController,
  keyboardType: const TextInputType.numberWithOptions(decimal: true),
  decoration: const InputDecoration(labelText: 'Tank capacity (L)'),
  validator: (value) {
    final parsed = double.tryParse(value ?? '');
    if (parsed == null || parsed <= 0) {
      return 'Enter a valid tank capacity';
    }
    return null;
  },
);
```

Choosing the Date

The date of a fill is selected with the Material date picker. Future dates are disabled because a fuel purchase cannot be recorded in advance.

```
final picked = await showDatePicker(
  context: context,
  initialDate: _date,
  firstDate: DateTime(2000),
  lastDate: DateTime.now(),
);
if (picked != null) {
  setState(() => _date = picked);
}
```

Saving the Log

When the form is valid, a FuelLog is created and handed to the provider. The provider writes it to SQLite and recalculates mileage and monthly expenses.

```
void _save() async {
  if (!(_formKey.currentState?.validate() ?? false)) return;

  final log = FuelLog(
    vehicleId: vehicle.id!,
    date: _date.toIso8601String().substring(0, 10),
    odometer: double.parse(_odometerController.text),
    fuelVolume: double.parse(_fuelController.text),
    totalCost: double.parse(_costController.text),
    isFullTank: _isFullTank,
  );

  await context.read<FuelProvider>().addFuelLog(log);
  if (mounted) Navigator.pop(context);
}
```

Submitting and Feedback

The submit button is disabled while a save is in progress, and the form shows a message when the operation fails. This prevents duplicate entries caused by repeated taps.

```
bool _saving = false;

FilledButton(
  onPressed: _saving ? null : _save,
  child: _saving
      ? const SizedBox(
          width: 18,
          height: 18,
          child: CircularProgressIndicator(strokeWidth: 2),
        )
      : const Text('Save Log'),
);
```

Output:

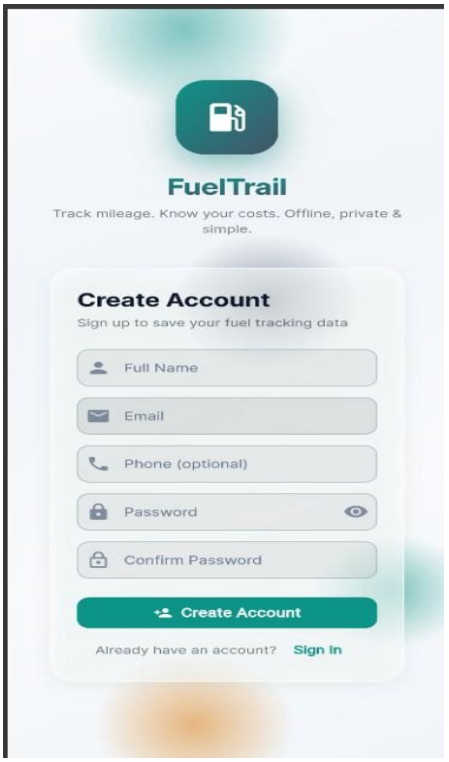


Figure 7: Validation messages shown by the registration form

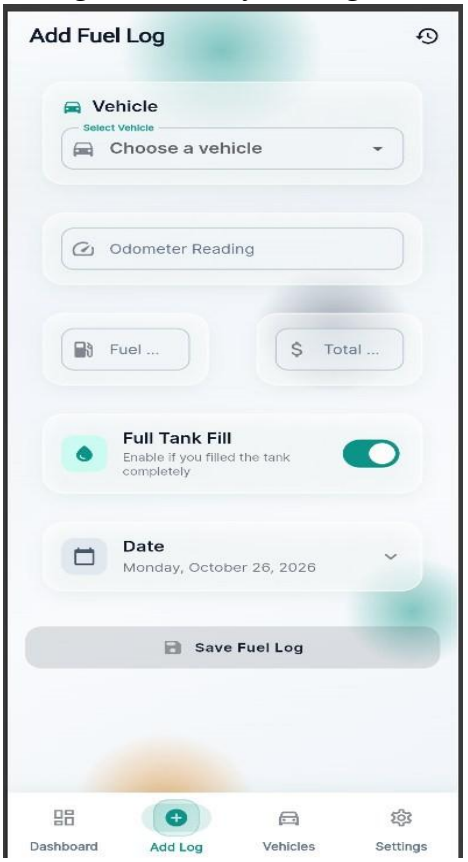


Figure 8: Add Fuel Log form with validation messages

16 Animations

Objective: To add subtle motion that improves feedback and makes the interface feel responsive.

Relation to Project: Animations are used on the dashboard cards, the mileage gauge, and the transitions between screens.

Description

The project uses three kinds of animation. Implicit animations such as AnimatedContainer and TweenAnimationBuilder handle simple value changes. An explicit AnimationController drives the press-scale effect on cards. Page transitions are provided by the theme.

Animated Cards

Each dashboard card scales down slightly while it is pressed, giving the user immediate tactile feedback.

```
class AnimatedCard extends StatefulWidget {
  final Widget child;
  final VoidCallback? onTap;

  const AnimatedCard({super.key, required this.child, this.onTap});

  @override
  State<AnimatedCard> createState() => _AnimatedCardState();
}

class _AnimatedCardState extends State<AnimatedCard> {
  bool _pressed = false;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTapDown: (_) => setState(() => _pressed = true),
      onTapUp: (_) => setState(() => _pressed = false),
      onTapCancel: () => setState(() => _pressed = false),
      onTap: widget.onTap,
      child: AnimatedScale(
        scale: _pressed ? 0.97 : 1.0,
        duration: const Duration(milliseconds: 120),
        curve: Curves.easeOut,
        child: widget.child,
      ),
    );
  }
}
```

Gauge Animation

The gauge fraction is animated from zero to its final value over about one second, so the needle sweeps into place when the dashboard opens.

```
return  TweenAnimationBuilder<double>(
  tween: Tween<double>(begin:  0, end: fraction),
  duration:  const  Duration(milliseconds:  1100),
  curve:  Curves.easeOutCubic,
  builder: (context, animatedFraction, _) {
    return  SizedBox(
      width:  width,
      height:  height,
      child: CustomPaint(
        painter: _MileageGaugePainter(
          fraction:  animatedFraction,
          // ... text painters and colours ...
        ),
      ),
    );
  },
);
```

Implicit and Explicit Animations

Flutter offers two approaches to motion. Implicit animations animate a property automatically when its value changes, while explicit animations give full control through an AnimationController.

Type	How it works	Examples in the project
Implicit	Animates when a property changes	AnimatedContainer, AnimatedScale, TweenAnimationBuilder
Explicit	Driven by an AnimationController	Press-scale feedback, page transitions

Animating a Container

An AnimatedContainer interpolates colour, size, radius, and padding whenever one of its values changes. The vehicle selector uses it to highlight the active vehicle.

```
AnimatedContainer(
  duration: const Duration(milliseconds: 220),
  curve: Curves.easeOut,
  padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
  decoration: BoxDecoration(
    color: selected
        ? theme.colorScheme.primaryContainer
        : theme.colorScheme.surface,
    borderRadius: BorderRadius.circular(14),
  ),
  child: Text(vehicle.name),
);
```

Page Transitions

The default Material transition is used for most routes. The add-log route uses a custom transition that fades and slides the new screen into place.

```
Navigator.push(
  context,
  PageRouteBuilder(
    transitionDuration: const Duration(milliseconds: 260),
    pageBuilder: (_, _, ____) => const AddLogScreen(),
    transitionsBuilder: (_, animation, _, child) {
      final curved = CurvedAnimation(
        parent: animation,
        curve: Curves.easeOutCubic,
      );
      return FadeTransition(
        opacity: curved,
        child: SlideTransition(
          position: Tween<Offset>(
            begin: const Offset(0, 0.06),
            end: Offset.zero,
          ).animate(curved),
          child: child,
        ),
      );
    },
  ),
);
```

Staggered Entry for Lists

Rows fade and rise into place with a delay based on their index, which draws the eye down the list without slowing the interface.

```
Widget _entry(int index, Widget child) {
return TweenAnimationBuilder<double>(
     tween: Tween(begin: 0, end: 1),
    duration: Duration(milliseconds: 250 + index * 60),
    curve: Curves.easeOut,
    builder: (context, value, _) {
      return Opacity(
        opacity: value,
        child: Transform.translate(
          offset: Offset(0, 18 * (1 - value)),
          child: child,
        ),
      );
    },
  );
}
```

Respecting Reduced Motion

When the operating system requests reduced motion, the duration is set to zero so that the interface remains comfortable to use.

```
final reduceMotion = MediaQuery.of(context).disableAnimations;
final duration = reduceMotion
    ? Duration.zero
    : const Duration(milliseconds: 250);
```

Output:



Figure 9: Dashboard Screen

17 Local Data Layer and API Integration

Objective: To provide a clean data layer that the rest of the application can use without knowing how storage works.

Relation to Project: Every screen reads and writes through the providers, which in turn use the database helper. There is no remote API; the "API" of this application is its local data layer.

Description

The application follows an offline-first design. The DatabaseHelper class exposes asynchronous create, read, update, and delete methods for users, vehicles, and fuel logs. It also exposes raw query helpers used by the database viewer. Because the helper is a singleton, the same database connection is shared across the application.

```
Future<int> insertFuelLog(int userId, FuelLog log) async {
  final db = await database;
  return await db.insert('fuel_logs', log.copyWith(userId: userId).toMap());
}

Future<List<FuelLog>> getFuelLogs({int? userId, int? vehicleId}) async {
  final db = await database;
  final where = <String>[];
  final args = <Object?>[];
  if (userId != null) {
    where.add('user_id = ?');
    args.add(userId);
  }
  if (vehicleId != null) {
    where.add('vehicle_id = ?');
    args.add(vehicleId);
  }
  final maps = await db.query(
    'fuel_logs',
    where: where.isEmpty ? null : where.join(' AND '),
    whereArgs: where.isEmpty ? null : args,
    orderBy: 'date ASC',
  );
  return List.generate(maps.length, (i) => FuelLog.fromMap(maps[i]));
}
```

Mileage Calculation

Mileage is calculated with the full-tank-to-full-tank method. The first full tank is used as a reference, the fuel consumed after it is accumulated, and the distance between consecutive full tanks is divided by that accumulated fuel.

```
final distance = log.odometer - refOdometer;
final mileage = distance > 0 ? distance / accumulatedFuel : 0.0;
final totalCost = accumulatedCost;
final costPerKm = distance > 0 ? totalCost / distance : 0.0;

records.add({
  'reference_date': refDate,
  'end_date': log.date,
  'start_odometer': refOdometer,
  'end_odometer': log.odometer,
  'distance': distance,
  'fuel_consumed': accumulatedFuel,
  'mileage': mileage,
  'cost_per_km': costPerKm,
  'total_cost': totalCost,
  'vehicle_id':  log.vehicleId,
});
```

Monthly Expenses

Monthly expenses are grouped directly in SQL, which keeps the aggregation fast and avoids loading every row into memory.

```
SELECT strftime('%Y-%m', date) as month, SUM(total_cost) as total_cost
FROM fuel_logs
WHERE vehicle_id = ? AND user_id = ?
GROUP BY strftime('%Y-%m', date)
ORDER BY month ASC
```

Built-in Database Viewer

The application includes a database manager that reads the schema from sqlite_master and shows the schema, DDL, and rows of each table. It also provides a small SQL console for running queries and updates.

```
Future<List<String>> getTableNames() async {
  final rows = await _db.rawQuery(
    "SELECT name FROM sqlite_master "
    "WHERE type='table' AND name NOT LIKE 'sqlite_%' ORDER BY name",
  );
  return rows.map((r) => r['name'] as String).toList();
}

Future<List<Map<String, dynamic>>> getTableSchema(String table) async {
  return await _db.rawQuery('PRAGMA table_info("$table")');
}
```

Atomic Writes with a Transaction

When a log is saved, the fuel row and the vehicle totals must stay consistent. A transaction guarantees that either both changes are applied or neither is.

```
Future<void> saveLog(int userId, FuelLog log) async {
  final db = await database;
  await db.transaction((txn) async {
    await txn.insert('fuel_logs', log.copyWith(userId: userId).toMap());
    await txn.rawUpdate(
      'UPDATE vehicles SET updated_at = ? WHERE id = ?',
      [DateTime.now().toIso8601String(),      log.vehicleId],
    );
  });
}
```

Running Ad-hoc Queries

The database viewer executes a read-only query and maps every returned row to a map that can be displayed in a table. Statements that modify data are routed through a separate method.

```
Future<List<Map<String, Object?>>> runQuery(String sql) async {
  final db = await database;
  return await db.rawQuery(sql);
}

Future<int> runStatement(String sql) async {
  final db = await database;
  return await db.rawUpdate(sql);
}
```

Error Handling in the Data Layer

Every database call is wrapped so that a failure is logged with a readable message. The error is rethrown so that the calling provider can inform the user.

```
try {
  final logs = await helper.getFuelLogs(userId: userId);
  return logs;
} catch (e) {
```

```
  debugPrint('Database error: $e');
  rethrow;
}
```

18 Testing and Debugging

Objective: To verify that the application behaves correctly and to diagnose problems when it does not.

Relation to Project: Widget tests cover the startup flow and the login form, while static analysis and manual testing cover the rest of the application.

Types of Testing

- **Unit Testing:** Tests individual functions such as the mileage calculation and the password hash.
- **Widget Testing:** Verifies that UI components render and respond correctly.
- **Integration Testing:** Tests an end-to-end flow such as opening the app, adding a vehicle, adding a log, and reading the dashboard.
- **Manual Testing:** Runs the app on an emulator or device and exercises each screen.

Widget Tests

The project includes widget tests that confirm the application starts on the login screen when no session exists, and that submitting an empty login form produces validation errors.

```
testWidgets('App loads and shows login when not signed in',
    (WidgetTester tester) async {
  SharedPreferences.setMockInitialValues({});
  final profileProvider = ProfileProvider();

  await tester.pumpWidget(
    MyApp(
      initialDarkMode: false,
      fuelProvider: FuelProvider(),
      profileProvider: profileProvider,
    ),
  );

  await tester.pumpAndSettle();

  expect(find.byType(MaterialApp), findsOneWidget);
  expect(find.byType(LoginScreen),  findsOneWidget);
});

testWidgets('Login form blocks submission when fields are empty',
    (WidgetTester tester) async {
  SharedPreferences.setMockInitialValues({});
```

```
  final profileProvider = ProfileProvider();
  await tester.pumpWidget(
    MyApp(
      initialDarkMode: false,
      fuelProvider: FuelProvider(),
      profileProvider: profileProvider,
    ),
  );
  await tester.pumpAndSettle();
  final signInButton = find.widgetWithText(FilledButton, 'Sign In');
  await tester.ensureVisible(signInButton);
  await tester.tap(signInButton);
  await tester.pumpAndSettle();

  expect(find.text('Enter your email'), findsWidgets);
  expect(find.text('Enter your password'), findsWidgets);
  expect(find.byType(LoginScreen), findsOneWidget);
  expect(profileProvider.isLoggedIn, isFalse);
});
```

Manual Test Cases

The following cases were executed manually on the emulator. Each case lists the steps and the expected result.

ID	Scenario	Steps	Expected result
T1	Register a user	Fill every field and submit	Home screen opens
T2	Mismatched passwords	Enter two different passwords	Error message appears
T3	Guest access	Tap Continue as guest	Dashboard opens
T4	Add a vehicle	Enter name, type, and capacity	Vehicle appears in the selector
T5	Add a fuel log	Enter odometer, fuel, and cost	Log appears in the history
T6	Compute mileage	Add two full tanks	Gauge shows a mileage value
T7	Monthly expenses	Add logs in two months	Two groups appear in the chart
T8	Toggle the theme	Switch to dark mode	Theme persists after restart
T9	Delete a log	Confirm the delete action	Row is removed from the list
T10	Inspect the schema	Open the database manager	Tables and DDL are listed

Running the Checks

Static analysis and the widget tests are run from the project root before every commit.

```
flutter analyze
flutter test
```

A clean run prints the following output.

```
Analyzing fuel_consumption_tracker...
No issues found!

00:03 +3: All tests passed!
```

Debugging

Debugging is performed with Flutter DevTools, `debugPrint` statements, and `try/catch` blocks around every asynchronous database or storage operation. Static analysis is run with `flutter analyze`, and formatting is kept consistent with `dart format`.

```
try {
  await profileProvider.load();
} catch (e) {
  debugPrint('Profile load error: $e');
}
```

19 Work Flow Of Screens

The following pages present the screen flow of the application. Each screen is shown in the order in which a user encounters it, beginning with authentication and ending with the database manager. Image placeholders are provided so that screenshots can be inserted in the same layout as the rest of this record.

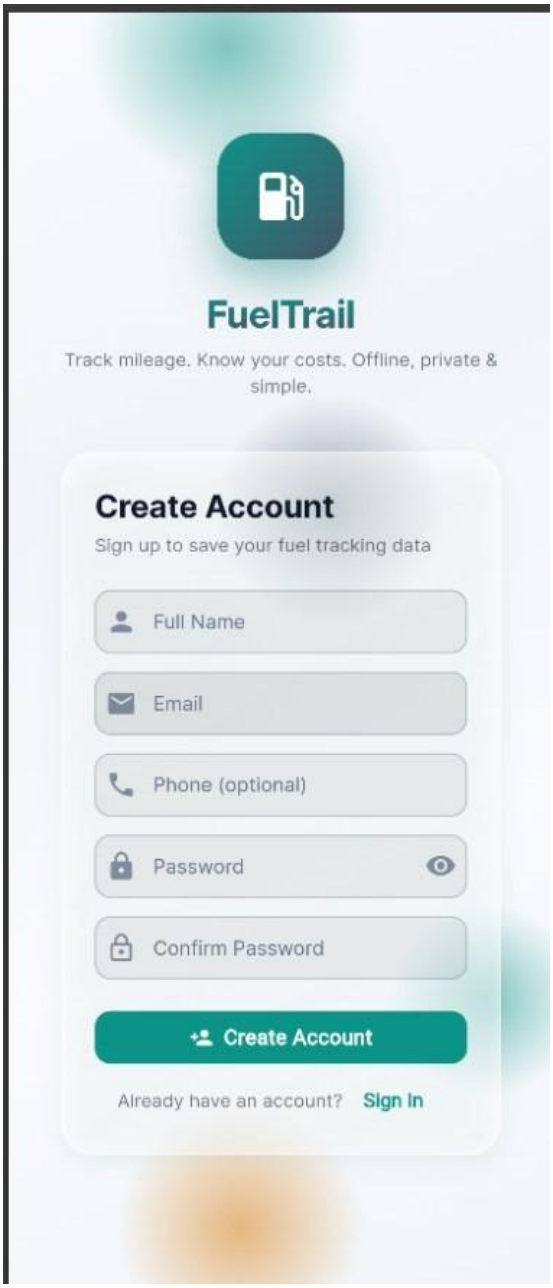


Figure 10: Register Screen

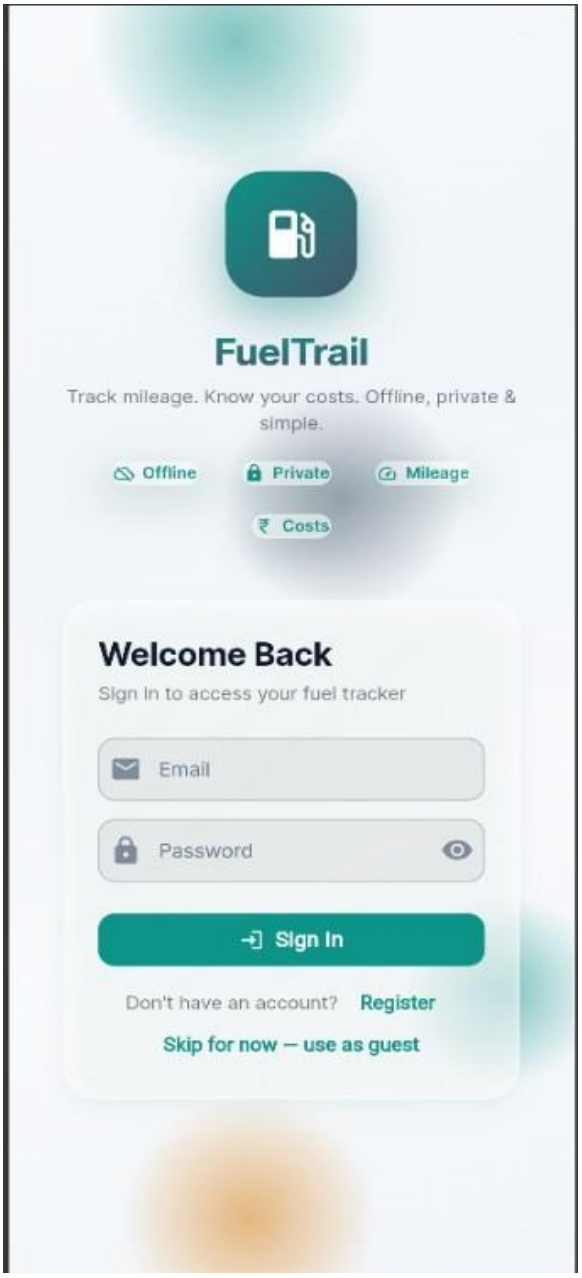


Figure 11: Login Screen

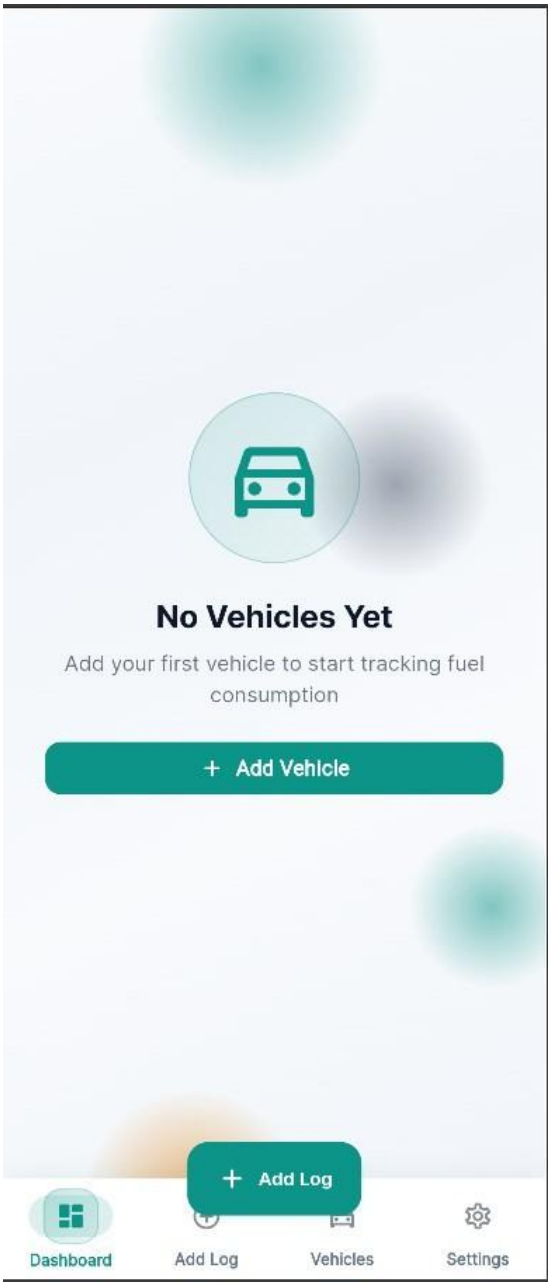


Figure 12: Dashboard Screen (empty state)



Figure 13: Dashboard Screen

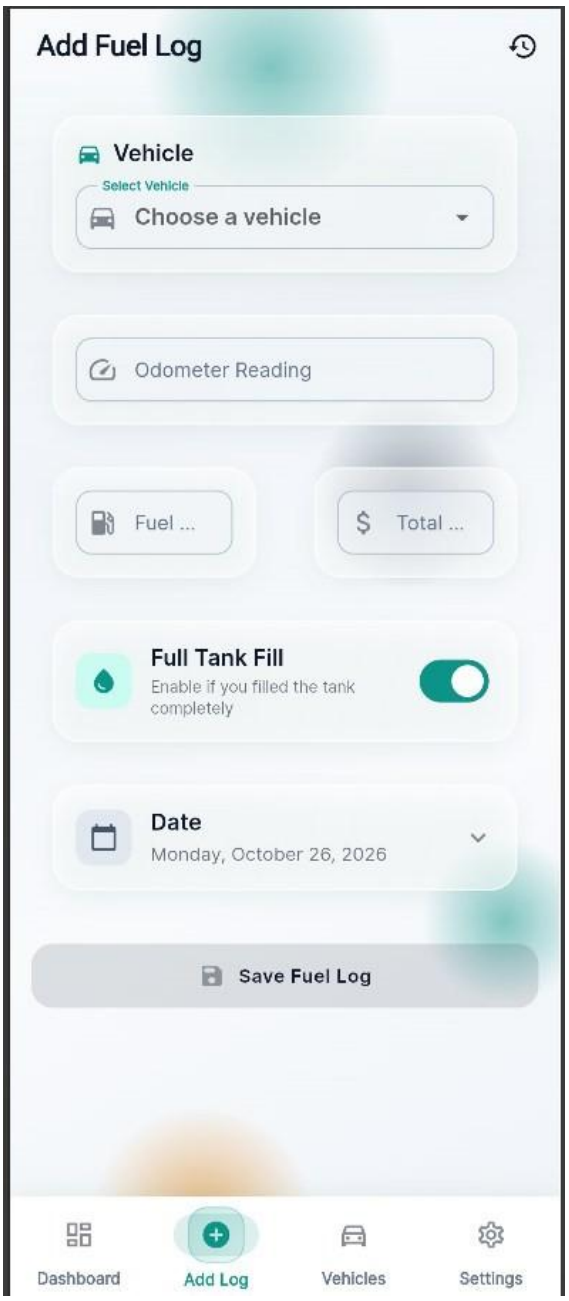


Figure 14: Add Log Screen



Figure 15: Vehicles Screen

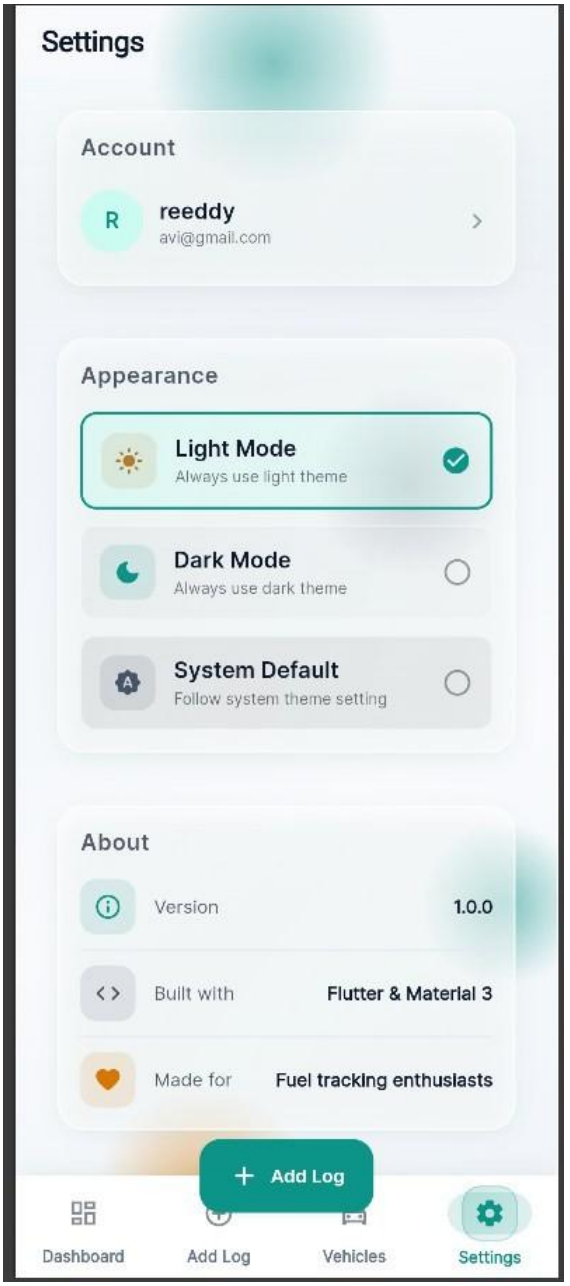


Figure 16: Settings Screen

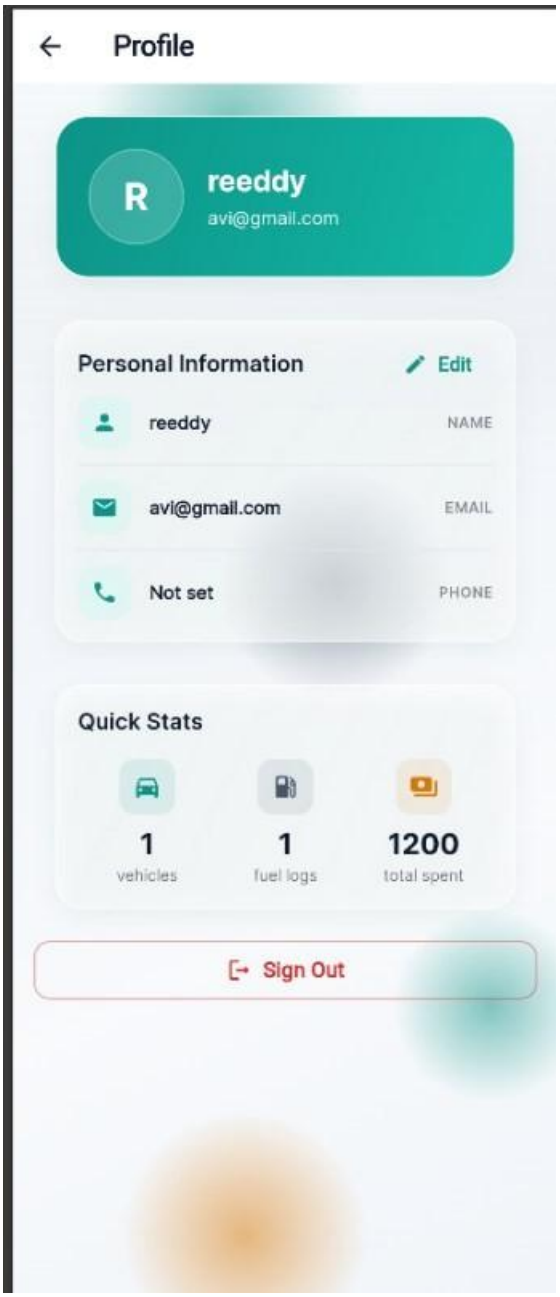


Figure 17: Profile Screen



Figure 18: Database Manager Screen

20 Conclusion

Through this project, core Flutter principles such as widgets, layouts, navigation, animations, forms, and state management were applied to build a complete, functional application. The Fuel Consumption Tracker demonstrates how these concepts combine in a real project with persistent local storage and a consistent visual identity.

The project also provided hands-on experience with SQLite schema design, migrations, indexing, and aggregation queries, as well as with offline authentication using salted password hashing. Building the custom mileage gauge and the glassmorphism interface strengthened the understanding of CustomPaint, Material 3 theming, and reusable widget design.

Because the application is offline-first, it remains usable without a network connection and keeps user data private on the device. The built-in database manager makes the data layer transparent and easy to inspect, which is valuable both for learning and for debugging. Overall, the project strengthened practical skills in Flutter and Dart and provided a solid foundation for building larger, data-driven mobile applications.

Learning Outcomes

- Designing a relational schema with foreign keys, indexes, and safe migrations.
- Structuring a Flutter application into models, providers, services, screens, and reusable widgets.
- Managing application state with Provider and persisting preferences with SharedPreferences.
- Building responsive layouts with MediaQuery, LayoutBuilder, and SafeArea.
- Implementing offline authentication with salted password hashing.
- Drawing custom graphics with CustomPaint and animating them smoothly.

Future Enhancements

The current version is complete for its intended purpose, and the following improvements are planned for later releases.

- Optional cloud backup so that data can be restored on a new device.
- Attaching a photograph of the receipt to each fuel log.
- Service reminders based on distance travelled and time elapsed.
- Exporting fuel history and expense reports to CSV and PDF.
- Support for multiple devices through an account synchronisation service.
- Predictive maintenance suggestions derived from mileage trends.